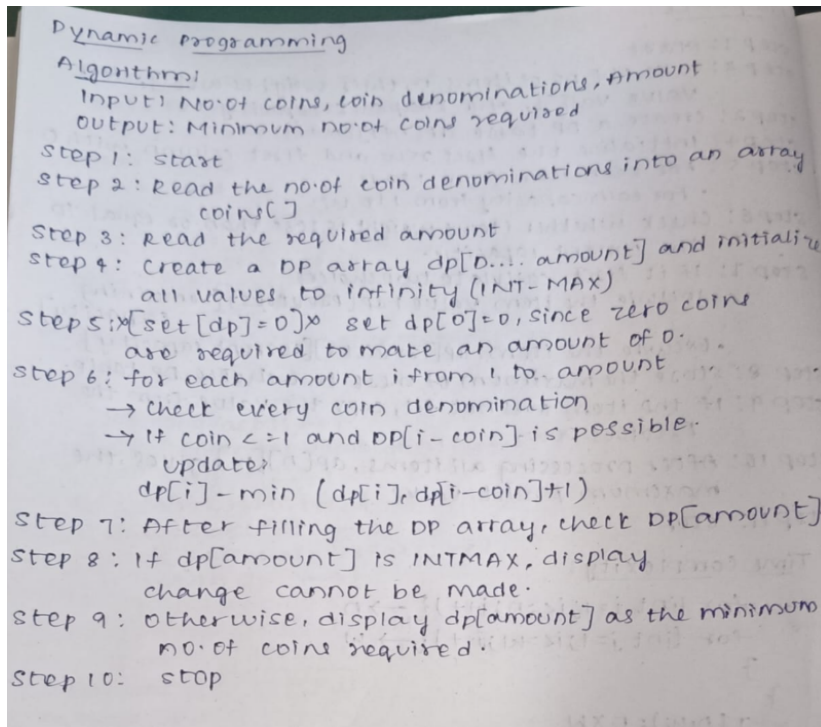


PRACTICAL -7

Aim: Implementation of making a change problem using dynamic programming.

Algorithm:



Dynamic programming
Algorithm:
Input: No. of coins, coin denominations, Amount
Output: Minimum no. of coins required

Step 1: start
Step 2: Read the no. of coin denominations into an array coins[]
Step 3: Read the required amount
Step 4: Create a DP array dp[0... amount] and initialize all values to infinity (INT- MAX)
Step 5: [set dp[] = 0] set dp[0] = 0, since zero coins are required to make an amount of 0.
Step 6: for each amount i from 1 to amount
 → check every coin denomination
 → If coin <= i and dp[i - coin] is possible.
 update
 dp[i] = min (dp[i], dp[i - coin] + 1)
Step 7: After filling the DP array, check dp[amount]
Step 8: If dp[amount] is INTMAX, display change cannot be made.
Step 9: Otherwise, display dp[amount] as the minimum no. of coins required.
Step 10: stop

program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n, amount;

    cout << "Enter number of coins: ";
    cin >> n;

    vector<int> coins(n);

    cout << "Enter the coin denominations:" << endl;
    for (int i = 0; i < n; i++) {
```

```
cin >> coins[i];
    }

cout << "Enter the amount: ";

cin >> amount;

// dp[i] = minimum number of coins needed to make amount i

vector<int> dp(amount + 1, amount + 1);

dp[0] = 0;

// Dynamic Programming

for (int i = 1; i <= amount; i++) {

    for (int j = 0; j < n; j++) {

        if (coins[j] <= i) {

            dp[i] = min(dp[i], dp[i - coins[j]] + 1);

        }

    }

}

if (dp[amount] > amount) {

    cout << "Amount cannot be made using given coins." << endl;

}

else {

    cout << "Minimum number of coins required: "

        << dp[amount] << endl;

}

return 0;
}
```

Output:

```
Enter number of coins: 7
Enter the coin denominations:
5
4
6
9
2
7
8
Enter the amount: 15
Minimum number of coins required: 2

=== Code Execution Successful ===
```

Time complexity:

Time complexity:

```
for (int i = 1; i <= amount; i++) { → amount
    for (int j = 0; j < n; j++) { → n
        if (coins[j] <= i) → 1
            dp[i] = min(...) → 1
    }
}
```

$T(n) = \text{amount} \times n$
 $T(n) = O(n \times \text{amount})$

Best case: The loops execute completely even for favourable input.

Average case: The same no. of iterations are performed for typical input.

Worst case: All amount & all coin denominations are checked.

Conclusion : Thus, the Making Change problem was successfully implemented using the Dynamic Programming technique. The algorithm efficiently finds the minimum number of coins required to make a given amount by breaking the problem into smaller subproblems and storing their results. This approach avoids repeated calculations and improves the efficiency compared to a simple recursive solution.